Mobile communication technologies have evolved rapidly from 3G to 4G, and now to 5G over the years, advancing from basic analog systems to high-speed digital communication [1]. Unlike previous generations, the fifth-generation mobile network (5G) targets higher data rates, lower latency, and massive connectivity. It also introduces advanced features such as cloud-native architecture and network slicing, which enable flexible deployment and customizable network functions for diverse service requirements [2-4].

As user traffic continues to grow and becomes increasingly unpredictable, inefficient resource allocation in 5G Core Networks can lead to degraded performance, higher costs, and unreliable service. Dynamic resource management is therefore crucial to maintaining service quality and optimizing performance. Autoscaling has become a key mechanism in modern 5G systems, allowing network functions to automatically scale up and down based on real-time traffic demands. This approach enhances performance, improves resource utilization, and preserves service quality under varying workloads [5].

This project implements and evaluates all three autoscaling algorithms on the same Kubernetes-based 5G testbed to compare their scaling behavior, reaction time, and decision accuracy under varying traffic conditions.

The remainder of this report is organized as follows: Section 2 provides relevant backgrounds on 5G core network architecture, Kubernetes and cloud-native deployment, and autoscaling strategies; Section 3 describes the methodology, including the experimental design and evaluation metrics; Section 4 details the implementation of the testbed and the three autoscaling approaches; Section 5 presents the experimental results; Section 6 discusses the findings and compares the performance of each approach; and Section 7 concludes the report and outlines directions for future work.

2. Background

2.1) 5G Core Network Architecture (x,6,a)

The 5G system architecture (5GS) introduces the fifth-generation core network (5GC), which is specified by 3GPP, enabling new service types *such as enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communications (URLLC), and massive Machine-Type Communications (mMTC).* The previous generations relied on point-to-point network architectures, where network functions were integrated into a unified system. To address these limitations, the 5GC adopts a Service-Based Architecture (SBA), where each network function operates as an independent microservice and communicates with other functions via standardized HTTP/2.0 APIs. This modular approach allows flexible deployment, independent scaling, and easier maintenance, making it well suited for cloud-native environments.

In a 5G network architecture, the User Equipment (UE) represents the end-user device, such as a smartphone or IoT sensor, which connects to the network through the Radio Access Network (RAN). The RAN consists of gNodeBs (gNBs), which serve as the 5G base stations responsible for managing the radio interface between the UE and the core network. Traffic from the UE is transferred through the gNB and forwarded into the 5G core network for processing and routing.

The 5G Core Network consists of several core network functions (NFs):

Access and Mobility Management Function (AMF) manages device registration, connection management, and mobility.

Session Management Function (SMF) is responsible for session management, IP address allocation, and policy enforcement.

User Plane Function (UPF) handles all user-plane data forwarding, including packet routing and traffic inspection.

Authentication Server Function (AUSF) performs authentication of UE devices during network registration.

Network Repository Function (NRF) provides service discovery, allowing network functions to discover other NFs and communicate via APIs.

Unified Data Management (UDM) stores and manages subscriber profile data including subscription information and authentication credentials.

Open5GS is an open-source project that implements the 5G core network functions based on 3GPP standards. Its modular design allows each network function to be containerized and deployed independently. UERANSIM is also an open-source simulator for the 5G UE and RAN, compatible with Open5GS, enabling end-to-end testing of 5G networks without requiring physical hardware.

2.2) Docker, Kubernetes, and Cloud-Native [y] [z] [a]

Cloud Native is an approach to building and running applications that take advantage of containerization, microservices, and automated orchestration [a]. Containerization is a virtualization approach that enables multiple applications to run on the host as isolated units known as containers. These containers not only can be deployed on the host but also can share the same resources by using Docker. Docker provides the foundation for creating, packaging, and distributing these containers. However, as the scale and complexity of the application increase, managing multiple containers in distributed infrastructure becomes more challenging, leading to the development of orchestration frameworks such as Kubernetes [y].

Kubernetes is an open-source orchestration platform that helps to automate the deployment, scaling, and management of containerized applications. In Kubernetes, those containers are organized into pods, which can be replicated mainly for redundancy and scalability. These pods are then organized and managed within a

cluster, which is a set of machines, known as nodes, working together to run containerized applications. Furthermore, Kubernetes continuously monitors these pods and ensures the desired state is maintained, handling Pod creation and destruction automatically [z].

2.3) Autoscaling Approaches

Autoscaling is an essential mechanism that provides the ability to dynamically scale vertically by increasing resources allocated to an existing instance, and horizontally by adding or removing instances in response to changes in workload [d]. This project focuses on the horizontal scaling as it aligns with the microservices-based design of the 5G Core Network, where adding or removing pod replicas of individual network function is more practical than resizing them.

Autoscaling algorithms can be categorized into two main types: Reactive and Proactive. Reactive algorithms respond to current system conditions and only scale resources after workload has changed, while proactive algorithms attempt to scale resources by considering future workload patterns [d]. This project compares three autoscaling algorithms, which are Horizontal Pod Autoscaler (HPA) as a reactive method, Autoregressive Integrated Moving Average (ARIMA) as a proactive statistical method, and Deep Q-Networks (DQN) as a proactive machine learning method.

2.3.1) Horizontal Pod Autoscaler (HPA)

The Horizontal Pod Autoscaler (HPA) is a built-in Kubernetes autoscaler and is widely used as an industry standard for reactive scaling. It is a threshold-based algorithm that monitors resource metrics, such as CPU utilization, and adjusts the number of Pod replicas when the observed metric exceeds or falls below the defined threshold [c].

2.3.2) Autoregressive Integrated Moving Average (ARIMA)

The Autoregressive Integrated Moving Average (ARIMA) is a proactive algorithm that uses time-series analysis to predict future resource utilization based on historical data. By forecasting upcoming workload, ARIMA allows scaling decisions to be made before workload changes occur, helping to avoid sudden spikes or drops in the number of Pod replicas [c][e].

2.3.3) Deep Q-Network (DQN)

The Deep Q-Network (DQN) is a proactive machine learning-based algorithm that uses reinforcement learning to learn a scaling policy from historical training data. The algorithm observes the current state of the system, such as CPU and memory utilization and the number of Pod replicas, before performing a scaling action. After the action is performed, the algorithm then receives a reward as an outcome determining whether the action improved or worsened the system performance, where the reward function is a scoring system that tells the agent how good or bad each scaling decision is at each step and allows it to learn from experience over time.

DQN also uses a neural network to estimate how good each action is and handle all possible state combinations. By repeating this process over time, the algorithm can learn which actions lead to better outcomes and adapt its scaling strategy as it gains more experience [b].

2.4) Monitoring and Evaluation

Monitoring is a critical part of managing containerized applications, as it provides visibility into the performance and health of the system. By continuously collecting and analyzing performance metrics, monitoring improves decision making for resource allocation and autoscaling [e]. In this project, Prometheus and Grafana tools, which are the most widely adopted, are used for this purpose in the Kubernetes environment.

Prometheus is an open-source monitoring platform that provides a multi-dimensional data model by collecting real-time metrics in the form of time series from data sources and storing them locally on a disk, identified by metric name and key/value pairs. Prometheus can automatically discover and monitor Pods and services, making it ideal for the Kubernetes environment. Prometheus uses a pull model over HTTP to collect real-time metrics and utilizes PromQL, which is Prometheus's built-in query language, to enable flexible queries on metrics including CPU utilization, memory usage, and network traffic [e] [f].

Grafana is an open-source analytics and visualization tool that is commonly used alongside Prometheus. Grafana queries various performance metrics from Prometheus and provides customizable dashboards to display graphical representations of metrics in real time. The combination of Prometheus and Grafana provides real-time insights into the application's performance and helps to identify and resolve issues proactively [g] [e].

Performance metrics are the measurements to determine when to scale pod replicas. These metrics can be categorized as internal and external metrics. Internal metrics, such as CPU utilization and memory usage, are collected directly from within the pods and are commonly used as the default scaling criteria. While internal metrics enable rapid responses to immediate resource needs, they may not fully capture the actual demand on a pod. In contrast, external metrics are sourced from outside the Kubernetes cluster and provide richer insights into application performance and user behavior, allowing better scaling decisions. Examples of external metrics include latency, queue length, and packets per second (PPS), which can measure the network traffic reaching the application. Using external metrics allows autoscaling to be more responsive to real-world traffic patterns rather than relying on internal metrics [h] [i].

3. Design and Implementation

3.1) System Architecture

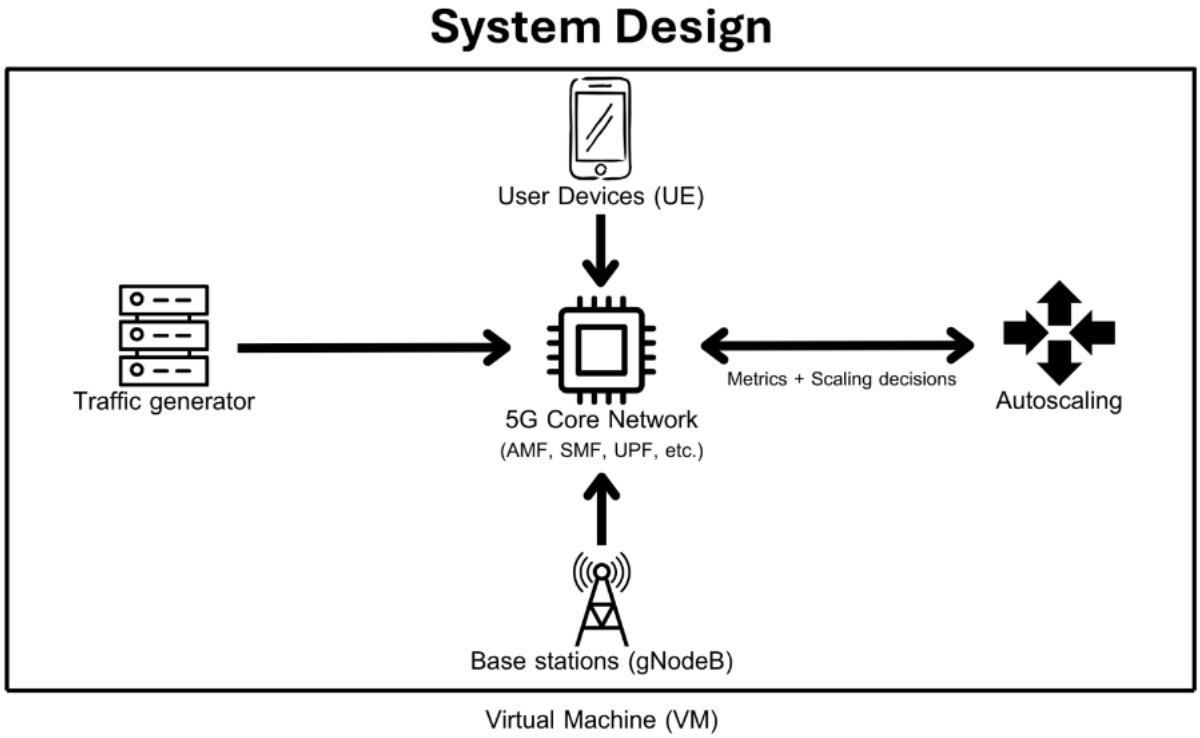


Figure X illustrates the overall system architecture designed for this project. The testbed consists of an Open5gs 5G core network, a simulated Radio Access Network (RAN) using UERANSIM, a MongoDB database for subscriber data, and Prometheus and Grafana for monitoring. Three autoscaling algorithms, including HPA, ARIMA, and DQN, were implemented and evaluated on UPF1, using packets per second (PPS) as the metric to trigger scaling actions. Traffic was generated through the simulated UEs at varying levels of demand to evaluate and compare the scaling behavior of each algorithm. The 5G core network is based on the open5gs-k8s project, which is an open-source Kubernetes deployment of the 5G core network [j] [k]. The autoscaling algorithms, custom metrics pipeline, and monitoring infrastructure were developed as the original contribution of this project.

3.2) Autoscaling Algorithms

HPA, ARIMA, and DQN autoscaling algorithms were implemented to evaluate different scaling strategies. These algorithms represent three distinct algorithm categories, including reactive, proactive, and machine-learning, allowing a comprehensive comparison across fundamentally different decision mechanisms.

HPA is selected as the baseline algorithm because it represents the standard Kubernetes autoscaling mechanism and is widely adopted in production environments. HPA operates scaling reactively by comparing the current observed PPS metric against a predefined target threshold to determine whether the scaling action is necessary. HPA is designed to scale up immediately when the traffic exceeds the threshold, while using a stabilization window to delay scale-down actions to prevent unnecessary replica removal during temporary dips.

ARIMA is selected as a proactive algorithm that forecasts future PPS from recent historical traffic data to trigger scaling decisions ahead of demand. However, since ARIMA cannot produce reliable forecasts without sufficient observations, it is designed to first collect PPS measurements during an initial warmup period before the forecasting and scaling process begins.

DQN is selected as a machine-learning-based approach that learns a scaling policy from past traffic data rather than relying on fixed rules. Unlike HPA and ARIMA, which can operate immediately, DQN requires training before live deployment. Therefore, several traffic datasets are first collected from the live testbed to train the agent offline, allowing it to develop a scaling policy efficiently before being evaluated against the actual workload.

3.3) Traffic Generator

To ensure a fair comparison, all three algorithms operate on the same PPS metric and target the same UPF1 deployment. A shared predefined threshold is applied across all three algorithms to consistently determine when scaling actions are necessary. Traffic is generated using iperf3, which is a network performance measurement tool that generates UDP traffic, a lightweight and connectionless protocol commonly used in real-time network applications, at a configurable bitrate for simulating realistic and varying network loads through the UPF. The traffic pattern is designed to be consistent, alternating between low-traffic and high-traffic phases to evaluate the scaling behavior of each algorithm under different workload conditions.

3.4) Metric Selection

Performance metrics are a crucial factor, as they directly determine when and how scaling actions are triggered. Therefore, selecting an appropriate scaling metric is essential, as a poorly chosen metric may result in delayed or unnecessary scaling, reducing the overall effectiveness of the autoscaling algorithm. Traditional metrics such as CPU and memory utilization are commonly used for general-purpose workloads. However, for network functions handling user traffic, such as the UPF, these metrics may not accurately reflect the actual network demand, as resource consumption does not always correlate directly with the volume of traffic being processed.

Initially, throughput (measured in Mbps) was considered as the scaling metric. However, throughput can vary depending on packet sizes, meaning that the same Mbps value could represent varying levels of demand. Therefore, packets per second (PPS) was selected as the scaling metric for this project instead because it directly represents the number of packets handled by the UPF and provides a more accurate indication of network load. An increase in PPS indicates that more traffic demand is being placed on the network function, making it a more meaningful trigger for scaling decisions compared to other metrics such as CPU utilization, memory usage, and throughput [i].

PPS is collected using Prometheus and exposed to Kubernetes as a custom metric, enabling all three autoscaling algorithms to use the same metric for a fair comparison. While ARIMA and DQN algorithms can query the PPS metric directly from Prometheus, the HPA requires the Prometheus Adapter to expose PPS as a Kubernetes external metric, since HPA can only read metrics through the Kubernetes metrics API.

3.5) **Monitoring**

To achieve real-time monitoring, Prometheus and Grafana are deployed within the Kubernetes cluster. Prometheus is responsible for scraping and storing the PPS metric from the UPF, which serves as the primary input for all three autoscaling algorithms. Grafana is connected to Prometheus as a data source and provides a custom dashboard to visualize UPF traffic and replica scaling behavior in real time. Additionally, the Prometheus Adapter is deployed as part of the pipeline to bridge Prometheus and the Kubernetes metrics API, enabling the HPA to be able to read the PPS metric for its scaling decisions.

4. Implementation

4.1) 5G Core Network Deployment

The 5G core network testbed was deployed on a Kubernetes cluster running on an Ubuntu virtual machine hosted on Oracle VirtualBox. Each Open5GS network function runs as a separate pod within the open5gs namespace, following a microservices architecture. MongoDB was deployed to serve as a database to store subscriber data. Network Attachment Definitions were configured using Multus CNI and mapped to OVS bridges, providing separate network interfaces for the N2, N3, and N4 reference points. The Open5GS core network was then deployed, including AMF, SMF, UPF, and NRF.

After that, subscriber data was added into MongoDB via the WebUI, allowing a UE to successfully register and establish PDU sessions with the core network. UERANSIM was then deployed as the simulated RAN, consisting of a gNB and a UE instance. The testbed was validated by performing a ping test through the uesimtun0 tunnel interface on the UE and confirming that the traffic was passing through the UPF by capturing packets on the ogstun interface using tcpdump.

4.2) Monitoring and Metrics

Prometheus and Grafana were installed and deployed together using the kube-prometheus-stack Helm chart, in which Helm is a Kubernetes package manager, within the Kubernetes cluster. Prometheus was configured to discover and scrape UPF metrics automatically through Kubernetes service discovery. A dedicated metrics service was created for the UPF to expose its metrics endpoint to Prometheus. The

scrape interval was set to 5 seconds to ensure fine-grained visibility into traffic changes.

The primary metric used throughout this project is "fivegs_ep_n3_gtp_indatapktn3upf", an Open5GS-specific counter that tracks the cumulative count of incoming GTP data packets on the N3 interface of the UPF. The rate of this counter was computed over a 30-second window to calculate the packets per second (PPS) value. If multiple UPF pods are running, their individual PPS values are summed into a single number, which serves as the scaling signal for all three autoscaling algorithms. A scaling threshold of 1,500 PPS was defined and applied consistently across all three autoscaling algorithms, *serving as the primary decision boundary for determining when to scale up or scale down the number of UPF replicas*.

Since the HPA cannot query Prometheus directly, the Prometheus Adapter was deployed separately via Helm and configured to expose the PPS metric as a Kubernetes external metric. A PromQL query was defined to compute the PPS rate and expose it under the metric name "upf1_n3_in_pps", making it accessible through the Kubernetes metrics API for the HPA to read.

A custom Grafana dashboard was created to visualize UPF traffic and autoscaling behavior in real time, providing a graphical view of the PPS metric alongside the number of active UPF replicas during the experiments.

4.3) HPA Implementation

The Kubernetes Horizontal Pod Autoscaler (HPA) was configured to scale UPF1 based on the "upf1_n3_in_pps" external metric. The minimum number of replicas was set to 1, and the maximum was set to 5 replicas. The scaling threshold was set to 1,500 PPS per replica. When the total number of incoming PPS exceeds the threshold, the HPA automatically scales up by adding one additional pod at a time with a 30-second period between consecutive scaling actions, allowing the system to stabilize before deciding whether another scaling action is needed. Since the scale-up stabilization window was set to 0 seconds, the HPA reacts immediately responding to increasing traffic without any delay.

To prevent unnecessary scaling during brief traffic drops, a scale-down stabilization window of 60 seconds was applied, meaning that the HPA must observe consistently low traffic for at least 60 seconds before removing a replica. Since all three autoscaling approaches share the same deployment target, the HPA must be disabled at the start of each ARIMA and DQN experiment and re-enabled upon completion, ensuring only one controller is active at any given time.

4.4) ARIMA Implementation

The ARIMA-based autoscaling algorithm was implemented as a Python script running as a live control loop, querying Prometheus every 5 seconds to obtain the current PPS value. Each metric reading was appended to a sliding window of the last 30 samples, which served as the input for the ARIMA model to predict the next incoming PPS value. To avoid insufficient data, a minimum warmup window of 10 samples was required before ARIMA predictions began. During the warmup period, the current PPS value was used directly for making scaling decisions.

An ARIMA model was configured with an order of (2,1,2), where the parameters indicate 2 autoregressive terms, 1 degree of differencing to achieve stationarity, and 2 moving average terms, controlling how the model analyzes recent traffic patterns to make predictions. At each control iteration, the ARIMA model learns from the last 30 PPS readings and forecasts the PPS value for the next 3 steps ahead (15 seconds), with the average of the forecasted values used to determine the desired replica count. The same 1,500 PPS per replica threshold was applied, with replicas bounded between 1 and 5. After each scaling action, a 30-second cooldown was applied before the next scaling decision could be made, preventing rapid unnecessary scaling actions. If the ARIMA model was unable to produce a forecast, the controller switched to an exponential moving average of the recent PPS values to ensure a scaling decision could still be made.

4.5 DQN Implementation

The DQN-based autoscaling algorithm was implemented as a Python script consisting of two phases. While the first phase is an offline training phase using historical PPS data, the second phase is a live control loop phase that applies the trained model from the first phase to make real-time scaling decisions.

Before deployment, the DQN agent was trained offline on historical PPS data collected from running the traffic generator to record multiple PPS readings as training data. The number of training files was determined by monitoring the epsilon value during the training phase. Epsilon is a probability value between 0 and 1 that controls how much the agent explores randomly versus relying on what it has already learned, where a lower value indicates the agent has learned most traffic patterns from the training data and is more confident in its decisions. After three training files, epsilon stabilized at around 0.01 and showed no further decrease, indicating that the agent had sufficiently learned the traffic patterns and additional training data would not improve it further.

The agent interacted with a simulated environment that replicated the UPF scaling behavior, where at each step it observed the current state and chose one of three scaling actions: scale up, scale down, or hold. The state provided to the agent at each step consisted of three values: the current PPS normalized by the threshold, the PPS trend, and the current number of replicas normalized by the maximum. The agent

was trained for 500 episodes per training file, resulting in 1,500 total training episodes, where each episode represents one full pass through the training data. A replay memory of 10,000 past experiences and a batch size of 32 were used to ensure stable and consistent learning. The target network was updated every 10 episodes to further stabilize the training process.

The agent was guided by a reward function that encouraged correct scaling decisions. A reward of +10 was given when the replica count matched the ideal count, while penalties were applied for over-provisioning and under-provisioning. A small bonus of +2 was given for scaling in the correct direction, and a penalty of -3 was applied for scaling in the wrong direction.

Once training was completed, the model was saved and deployed in live mode with a PPS threshold of 1,500. The controller queried Prometheus every 5 seconds and built the current state, and the trained agent selected the best scaling action. A cooldown of 30 seconds was applied between consecutive scaling actions to prevent rapid unnecessary scaling.

4.6) Traffic Generator

The traffic generator was used to simulate realistic network traffic through the 5G core in order to test and evaluate the autoscaling algorithms. Traffic was generated using **iperf3**, a network performance tool that generates UDP packets from the UE pod through the UPF to a designated data network server at a configurable bitrate and packet size. The traffic follows a structured pattern consisting of five phases: an initial 30 second idle phase, a low traffic phase at 10 Mbps lasting 60 seconds, a second idle phase for 30 seconds, a high traffic phase at 40 Mbps lasting 120 seconds, and a final idle phase for 120 seconds. The low traffic phase is designed to observe whether the autoscaling algorithms maintain a stable replica count below the threshold, while the high phase is designed to trigger scale-up actions. The final idle phase observes whether the PPS metric drops to zero, and a scale-down action is subsequently triggered for each algorithm.

For collecting training data for the DQN agent, the traffic pattern was repeated multiple times, with each run recording PPS readings from Prometheus every 2 seconds into a CSV file. Three training files were collected in total, with a 15-second gap applied between runs to allow the environment to fully reset before the next recording. The collected data was used for offline training of the DQN algorithm prior to live deployment.

5. Analysis

All three autoscaling algorithms were evaluated under identical conditions using the same traffic pattern, metric, and deployment configuration, with each algorithm

tested independently on the UPF1 deployment. The traffic pattern alternated between low traffic at 10 Mbps and high traffic at 40 Mbps, separated by idle phases with no traffic. Across all three algorithms, the scaling threshold was set to 1,500 PPS per replica, with the minimum number of replicas set to 1 and the maximum to 5. The ideal replica count at any given time was determined by dividing the current PPS by the 1,500 PPS threshold. To ensure a fair comparison, all three algorithms were configured with a 30-second cooldown period between consecutive scaling actions, preventing unnecessary rapid oscillation and ensuring each scaling decision had time to take effect before the next one was made.

5.1 HPA Analysis

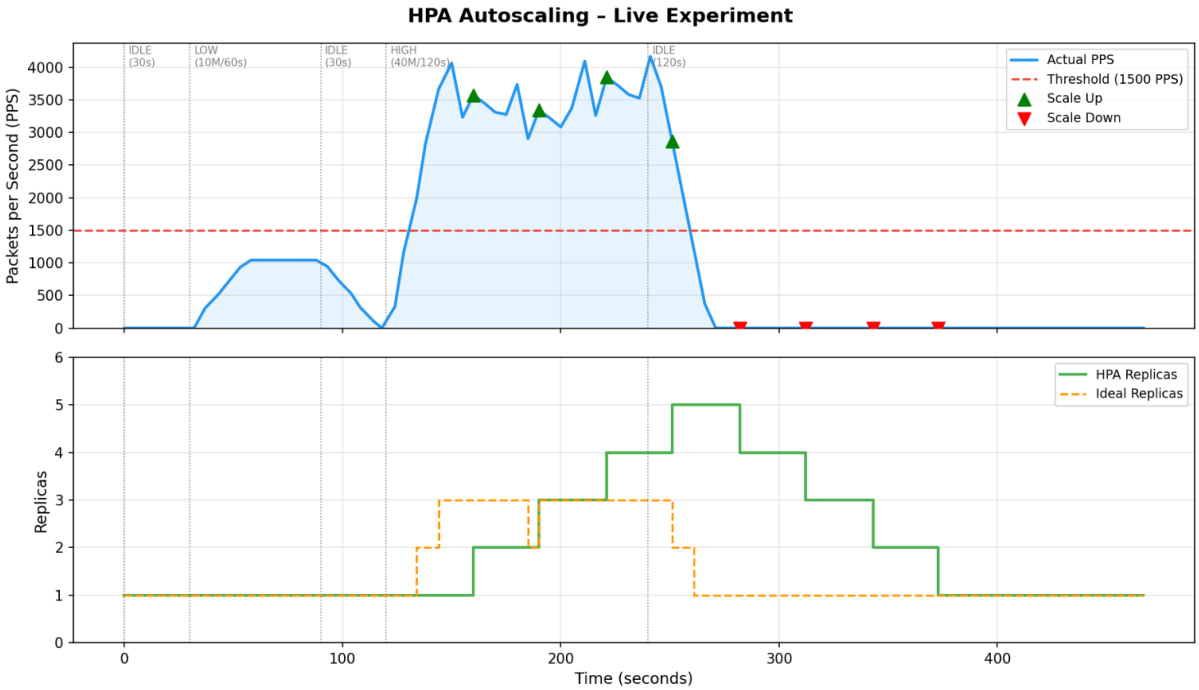


Figure...

Figure ... presents the results of the HPA autoscaling experiment. At the beginning, the PPS remained at zero during the initial idle phase before increasing to around 1,000 PPS during the low traffic phase, remaining below the threshold. The HPA correctly maintained one replica without triggering any scaling actions throughout the first three phases.

During the high traffic phase, the PPS rose dramatically to around 4,000 PPS, exceeding the threshold and triggering a series of scale-up actions. The HPA incrementally scaled up every 30 seconds from 1 to 5 replicas by adding one replica at a time. A notable characteristic observed during this phase is that the PPS displayed moderate fluctuations, which can be attributed to the scaling behavior. As each new replica was added, the incoming traffic was redistributed across all active replicas, causing a temporary drop in per-pod PPS before recovering as traffic continued to arrive. Despite this, the HPA continued scaling up until it reached 5 replicas, exceeding the ideal replica count of 3, indicating over-provisioning.

Once the traffic dropped during the final idle phase, the HPA scaled down gradually from 5 to 1 replica and remained at 1 replica until the end of the experiment. Overall, the HPA demonstrated reliable and predictable reactive scaling behavior, correctly responding to traffic changes based on the observed PPS metric. However, its purely reactive nature resulted in a delayed response to rising traffic and over-provisioning during the high traffic phase, as it could only react after the threshold had already been exceeded.

5.2) ARIMA Analysis

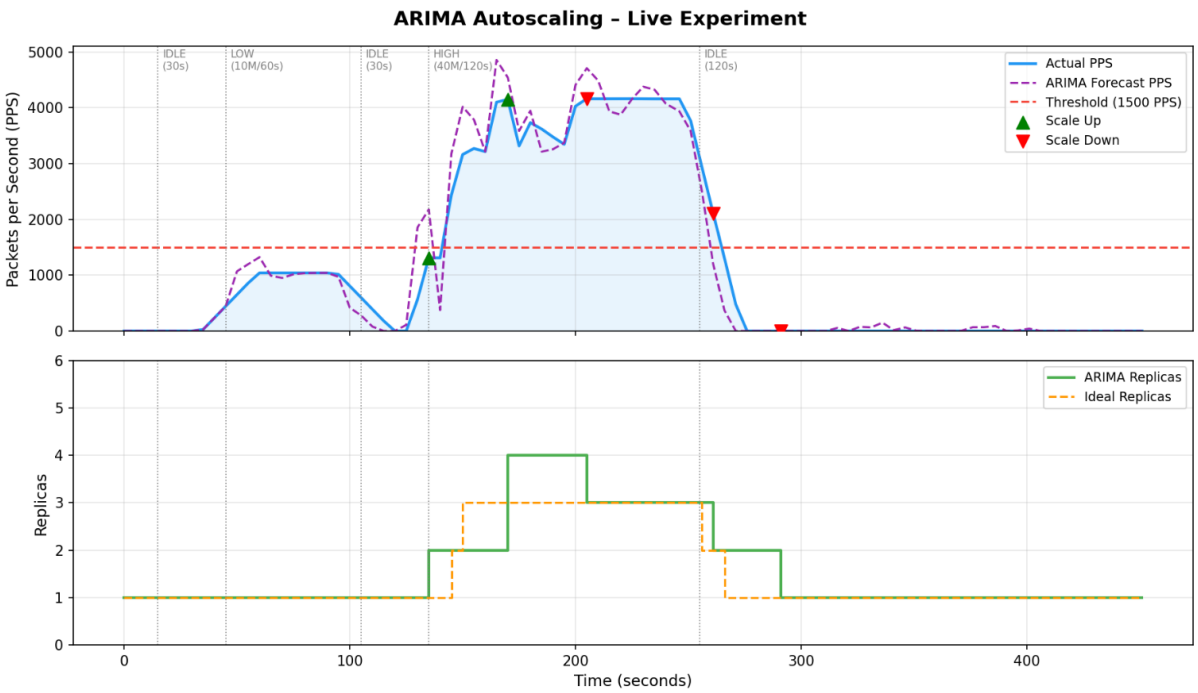


Figure...

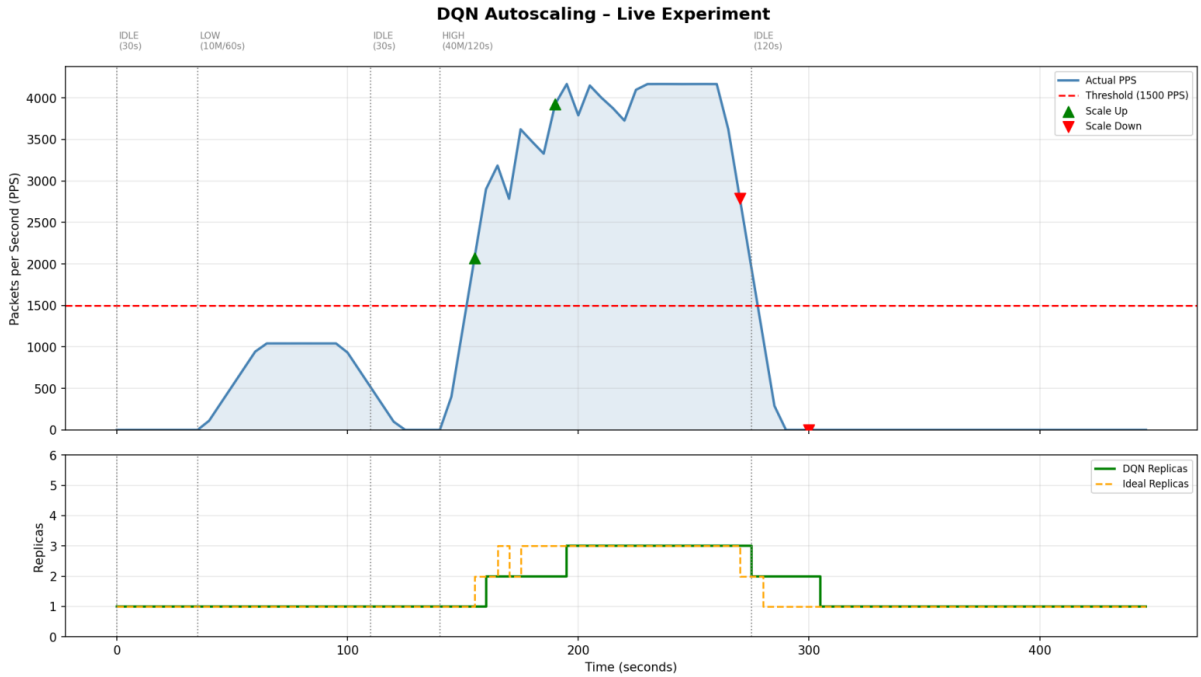
Figure ... shows how the ARIMA algorithm responded to the different traffic phases throughout the live experiment. Unlike HPA, ARIMA operates by forecasting future PPS values based on recent historical observations rather than reacting to the current metric directly. *During the initial idle and the early low traffic phases, ARIMA was still collecting sufficient observations to build its forecasting capability, known as the warmup period, where the forecast closely tracked the actual PPS values before starting to forecast reliably.* As the PPS remained below the 1,500 PPS threshold throughout the first three phases, ARIMA correctly maintained only one replica with no scaling action triggered.

When the high traffic phase began, the forecasted PPS increased remarkably and crossed the 1,500 PPS threshold ahead of the actual PPS, allowing ARIMA to trigger scale-up actions earlier than a purely reactive approach such as HPA. ARIMA scaled up from 1 to 4 replicas, overshooting the ideal replica count of 3 due to the forecast overestimating the incoming traffic. *However, in the middle of the high phase, ARIMA corrected itself by scaling back down to 3 replicas, realigning with the ideal replica*

count. The scaling actions were also limited by a 30-second cooldown period, which contributed to the stepwise nature of the replica changes.

In the final idle phase, after the traffic dropped below the threshold, ARIMA scaled down from 3 to 1 replica and held at 1 replica for the rest of the experiment. Overall, ARIMA demonstrated improved scaling accuracy over the HPA, with the proactive nature of its forecasting reducing over-provisioning. However, the sensitivity of the model to sudden traffic fluctuations resulted in occasional forecast overshoots, highlighting a key limitation of time-series based forecasting under highly variable traffic conditions.

5.3) DQN Analysis



Figure

Figure ... illustrates the behavior of the DQN algorithm across the live experiment traffic phases. DQN determines scaling decisions by learning from past traffic data, selecting scaling actions based on the current system state and reward function to achieve the best outcome. Throughout the first three phases, the PPS stayed below the 1,500 PPS threshold, and DQN maintained the replica count at one without triggering any scaling actions.

During the high traffic phase, the PPS grew significantly above the 1,500 PPS threshold, triggering a scale-up action from 1 to 2 replicas. As the traffic continued to climb, DQN triggered a second scale-up action, bringing the replica count up to 3, precisely matching the ideal capacity. *The 30-second cooldown period between scaling actions prevented DQN from reaching the target in a single step, introducing a slight delay in the scale-up response* compared to the ideal replica count. *Despite this, DQN achieved the target of 3 replicas without any overshooting, demonstrating more precise scaling behavior than ARIMA.*

Once the traffic fell below the threshold during the final idle phase, DQN stepped down from 3 to 1 replica in two steps, before remaining steady at 1 replica until the end of the experiment. Overall, DQN proved to be the most accurate autoscaling algorithm in terms of replica count compared to HPA and ARIMA, closely tracking the ideal replica capacity throughout the experiment. However, the 30-second cooldown introduced a slight delay in scaling decisions, representing a trade-off between avoiding rapid scaling fluctuations and achieving timely responsiveness.

6. Discussion

6.1) Comprehensive Analysis

This section presents a comparative analysis of the three autoscaling algorithms evaluated in this experiment: HPA, ARIMA, and DQN. The comparison is based on four observed criteria derived from the replica count behavior recorded during the live traffic experiment, including category, scaling accuracy, over-provisioning, and replica stability.

Criterion	HPA	ARIMA	DQN
Scaling Accuracy	Low	Medium	High
Over-provisioning	High	Moderate	None
Replica Stability	Low	Medium	High

As shown in the table above, DQN demonstrated the strongest overall performance across all three criteria, precisely matching the ideal replica count without any over-provisioning and maintaining stable replicas throughout the experiment. ARIMA showed moderate performance, with its proactive forecasting allowing it to self-correct after a brief overshoot, though its sensitivity to traffic fluctuations introduced instability. HPA is purely reactive, consistently over-provisioned, and results in the lowest scaling accuracy among the three algorithms.

6.2) Trade-off

Each autoscaling approach represents a different trade-off between simplicity, accuracy, and complexity. HPA, as a reactive approach, is the simplest to implement and requires no historical data or training. However, its reliance on real-time metric thresholds means it can only respond after a scaling condition has already occurred, making it inherently slow to adapt to rapid traffic changes. This simplicity comes at the cost of scaling accuracy and efficiency, as demonstrated by its over-provisioning behavior during the high traffic phase.

ARIMA, as a proactive approach, improves upon HPA by forecasting future traffic based on historical observations, allowing it to anticipate scaling needs before the threshold is exceeded. However, this comes with increased implementation

complexity and a dependency on sufficient historical data to produce reliable forecasts. Additionally, ARIMA's sensitivity to short-term traffic fluctuations can lead to forecast inaccuracies, resulting in occasional overshoots that require self-correction.

DQN, as a learning-based approach, offers the most advanced scaling behavior by learning an optimal scaling policy through interaction with the environment. This allows DQN to make more informed and precise scaling decisions compared to both HPA and ARIMA. However, DQN requires significantly higher implementation complexity, as it requires a training phase before deployment and careful tuning of the reward function and model hyperparameters. Furthermore, the quality of DQN's scaling decisions is heavily dependent on the representativeness of the training data, meaning that if the live traffic pattern differs significantly from the training data, its performance may degrade.

6.3) Limitations

Algorithm Limitations

HPA is limited by its purely reactive nature, meaning that it is unable to anticipate traffic changes and will always respond after a scaling condition has already occurred. This makes it inherently unsuitable for workloads with rapidly changing traffic patterns.

ARIMA assumes that future traffic patterns follow a statistically predictable trend based on recent observed traffic data to generate short-term forecasts. Additionally, ARIMA's performance is sensitive to the quality and quantity of recent data available for the model to make its predictions. Any sudden or irregular spike that deviates from the recent trend may result in an inaccurate forecast.

DQN requires a training phase before deployment, meaning it cannot be used in environments where no historical traffic data is available. Additionally, its scaling decisions are heavily dependent on the representativeness of the training data. Therefore, if the live traffic pattern differs significantly from what the model was trained on, its scaling performance may degrade, which represents a hard limitation in highly dynamic or previously unseen traffic scenarios.

Experiment Limitations

The experiment was conducted in a controlled environment using a synthetic and predefined traffic pattern, which may not fully reflect the complexity and unpredictability of real-world traffic. The evaluation was also limited to a single traffic pattern, meaning the results may not generalize to other workload profiles such as gradual ramp-ups, flash crowds, or periodic bursts. Furthermore, the maximum replica count was capped at 5, which limits the ability to observe how each algorithm behaves under more extreme traffic conditions.

7. Conclusion and Future Work

This report presents a comparative evaluation of three autoscaling algorithms, including HPA, ARIMA, and DQN, deployed in a Kubernetes environment under a controlled live traffic experiment. The aim is to determine which algorithm performs the best, achieving the most effective scaling accuracy for containerized applications in response to changing traffic demand.

The results demonstrated that DQN achieved the strongest overall autoscaling performance, with precise scaling actions to handle different traffic phases efficiently without any over-provisioning and maintaining stable scaling behavior. ARIMA showed decent performance, with its proactive forecasting allowing it to anticipate upcoming traffic changes. However, ARIMA's sensitivity to short-term traffic fluctuations led to a brief overshoot before self-correcting. While HPA is the simplest algorithm, it demonstrated the lowest scaling accuracy among the three algorithms due to its threshold-based reactive nature, which caused scaling delays and consistent over-provisioning.

In the future, we plan to evaluate the autoscaling algorithms under real-world traffic conditions by testing with a real SIM card and a smartphone, which would provide a more realistic and diverse range of traffic patterns. Additionally, exploring other autoscaling algorithms, such as a hybrid approach that combines ARIMA's proactive forecasting with DQN's learned scaling policy, could be a promising direction for further research to improve autoscaling performance and scaling accuracy.

References

[x] A. Vetsos, D. Uzunidis, P. Papadopoulos, and P. Karkazis, "Comparative analysis of three open-source 5G simulation tools," in *Proc. 28th Pan-Hellenic Conf. Progress in Computing and Informatics (PCI '24)*, New York, NY, USA, 2025, pp. 107–113.

[6] G. Brown *et al.*, "Service-based architecture for 5G core networks," *Huawei White Paper*, vol. 1, 2017.

[a] S. Imadali and A. Bousselmi, "Cloud Native 5G Virtual Network Functions: Design Principles and Use Cases," in *2018 IEEE 8th International Symposium on Cloud and Service Computing (SC2)*, 2018, pp. 91–96. doi: 10.1109/SC2.2018.00019

[y] R. T. Avireneni, S. H. Koneru, N. K. K. Reddy Yelkoti, S. Y. Khaga, and S. Nelavelli, "Cloud Orchestration with Kubernetes/Docker," *American International Journal of Computer Science and Technology*, vol. 4, no. 1, pp. 24–34, Jan. 2022. doi: 10.63282/3117-5481/AIJCST-V4I1P103

[z] J. Shah and D. Dubaria, "Building Modern Clouds: Using Docker, Kubernetes & Google Cloud Platform," in *2019 IEEE 9th Annual Computing and Communication Workshop and Conference (CCWC)*, 2019, pp. 0184–0189. doi: 10.1109/CCWC.2019.8666479

[b] D. Lee, J.-H. Yoo, and J. W.-K. Hong, "Deep Q-Networks based Auto-scaling for Service Function Chaining," in *2020 16th International Conference on Network and Service Management (CNSM)*, 2020, pp. 1–9. doi: 10.23919/CNSM50824.2020.9269107
[c] N. S. Joshi, R. Raghuwanshi, Y. M. Agarwal et al., "ARIMA-PID: container auto scaling based on predictive analysis and control theory," *Multimedia Tools and Applications*, vol. 83, pp. 26369–26386, 2024. doi: 10.1007/s11042-023-16587-0

[d] S. Alharthi, A. Alshamsi, A. Alseiari, and A. Alwarafy, "Auto-Scaling Techniques in Cloud Computing: Issues and Research Directions," *Sensors*, vol. 24, no. 17, p. 5551, 2024. doi: 10.3390/s24175551

[e] T. Abirami, S. Mapari, P. Jayadharshini, L. Krishnasamy, and R. R. Vigneshwaran, "Streamlined Deployment and Monitoring of Cloud-Native Applications on AWS with Kubernetes Prometheus Grafana," in *2023 International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT)*, 2023, pp. 1149–1155, doi: 10.1109/ICAICCIT60255.2023.10465818.

[f] N. Sukhija and E. Bautista, "Towards a Framework for Monitoring and Analyzing High Performance Computing Environments Using Kubernetes and Prometheus," in *2019 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI)*, Leicester, UK, 2019, pp. 257–262, doi: 10.1109/SmartWorld-UIC-ATC-SCALCOM-IOP-SCI.2019.00087.

[g] K. Pai and B. K. Srinivas, "Enhanced visibility for real-time monitoring and alerting in Kubernetes by integrating Prometheus, Grafana, Loki, and Alerta," *International Journal of Scientific Research in Engineering and Management*, vol. 8, pp. 1–5, 2024.

[h] H. Mehra and S. Goswami, "Comparative Analysis of Kubernetes Autoscaling Using Internal and External Metrics," in *2025 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT)*, 2025, pp. 1–6, doi: 10.1109/CONECCT65861.2025.11306504.

[i] V. Millnert and J. Eker, "HoloScale: horizontal and vertical scaling of cloud resources," in *2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC)*, 2020, pp. 196–205, doi: 10.1109/UCC48980.2020.00038.

[j] N. Saha, N. Shahriar, M. Sulaiman, N. Limam, R. Boutaba, and A. Saleh, "Monarch: Monitoring Architecture for 5G and Beyond Network Slices," *IEEE Transactions on Network and Service Management*, vol. 22, no. 1, pp. 777–790, 2025, doi: 10.1109/TNSM.2024.3479246.

[k] "Open5gs-k8s Github." Accessed: Sep. 2, 2024. [Online]. Available: https://github.com/niloysh/open5gs-k8s